

Elaborato per l'esame di
Architetture Dati

Progetto Architetture Dati - Confronto scalabilità tra DBMS relazionale e non relazionale

*Valutazione empirica della scalabilità in ambiente distribuito di PostgreSQL e
MongoDB*

CANDIDATO
Fabio Compagnoni
matr. 944954

DOCENTE
Prof. Andrea Maurino

1 Introduzione

Questo elaborato valuta empiricamente se e quando un DBMS distribuito sia adatto a un carico applicativo reale di tipo HR/retributivo (ditte, dipendenti, contratti, presenze e cedolini paga in regime *multitenant*), confrontando due sistemi che rappresentano filosofie opposte di gestione dei dati:

- **PostgreSQL + Citus:** un DBMS *relazionale* reso realmente distribuito dall'estensione Citus (un coordinator e N worker), con schema rigido, SQL completo, transazioni ACID e consistenza forte;
- **MongoDB:** un DBMS *documentale* (sharded cluster), con schema flessibile, dati organizzati in documenti annidati e distribuzione automatica tramite il balancer.

I due sistemi appartengono alla stessa classe rispetto al teorema CAP (entrambi **CP**) ma hanno modelli di dati molto diversi: questo permette di capire quando ciascun modello rappresenta bene questo tipo di dati e come si comportano in ambiente distribuito.

1.1 Contesto e motivazione

La scelta del dominio non è casuale: nasce da una collaborazione lavorativa con un'azienda di consulenza HR, tra i cui servizi principali c'è la creazione e l'elaborazione dei cedolini paga. Partendo da **file XML di paghe reali**, è sorta una domanda concreta: come modellare al meglio questi dati, sia con un database *relazionale* sia con uno *documentale* (non relazionale). In un contesto realistico con **migliaia di dipendenti e di aziende**, si tratta poi di capire quale dei due sistemi si comporti meglio e quindi quale convenga implementare in produzione.

Prima di avviare i test sono stati studiati i fondamenti teorici dei due sistemi e lette le rispettive documentazioni ufficiali: il modello dei dati, il linguaggio di interrogazione e il modo in cui affrontano scalabilità e transazioni concorrenti. Solo dopo si è passati alla verifica sperimentale. L'elaborato segue questa struttura: una parte teorica (Capitolo 2) e una sperimentale su un cluster reale (Capitoli 4 e 5), in cui si confronta quanto affermato in teoria con i dati misurati. Per il vincolo GDPR i dati reali non sono utilizzabili direttamente: il dataset è quindi sintetico ma *calibrato* sulla forma dei cedolini reali (Sezione 2.2).

1.2 Requisiti e criteri di valutazione

Il lavoro è organizzato attorno ai requisiti richiesti. Occorre disporre di un dataset ampio (o reso artificialmente grande) da interrogare con query a complessità variabile su entrambi i sistemi, e valutare *quando il modello modella bene* il dominio (Capitolo 3). Vanno poi verificati la gestione dei cambiamenti di schema in corso d'opera (Sezione 3.5), il comportamento in ambiente distribuito in lettura, scrittura (transazioni) e in presenza di guasti, e la scalabilità nei due regimi: a parità di dati aggiungendo nodi, e a parità di nodi aumentando il volume dei dati. Questi ultimi aspetti sono misurati sperimentalmente (Capitoli 4 e 5).

1.3 Approccio e struttura

Il metodo è sperimentale: entrambi i sistemi sono stati installati su un cluster di 5 macchine virtuali in cloud, alimentati con lo *stesso dataset* sintetico (calibrato su cedolini reali) e

sollecitati con una suite di query identica nella semantica. Per ogni prova si sono raccolte latenza, throughput e uso di risorse (CPU, memoria, I/O di rete e disco) *di ciascun nodo*, così da osservare non solo *quanto* costa un'operazione, ma *come* il lavoro si distribuisce nel cluster.

2 Dominio, fondamentali e scelte progettuali

Prima di progettare le basi di dati e la campagna sperimentale sono state prese alcune decisioni di fondo: su quale dominio applicativo lavorare, con quali dati, quali due sistemi confrontare e sulla base di quali concetti teorici. Questo capitolo le motiva e richiama i fondamentali necessari a interpretare i risultati.

2.1 Il dominio HR

Il dominio è quello della gestione del personale e delle retribuzioni (*payroll*): ditte, dipendenti, contratti, presenze e **cedolini paga**, in regime **multitenant**. Oltre all'origine reale già descritta (Sezione 1.1), è particolarmente adatto a mettere a confronto i due modelli di dati per alcune sue proprietà strutturali:

- **Multitenancy**: l'attributo `tenant_id` (la ditta) è la chiave di partizionamento naturale su entrambi i sistemi, pattern canonico dei sistemi distribuiti multitenant;
- **Contrasto tra i modelli**: il cedolino si presta a essere rappresentato come un unico documento (intestazione, voci, ratei, contributi, addizionali), ma sotto ha una struttura relazionale fatta di anagrafiche e cataloghi condivisi; permette quindi di osservare in quali casi conviene l'uno o l'altro modello;
- **Evoluzione di schema con una storia reale**: obblighi come la trasparenza retributiva sono un cambiamento concreto su un sistema già popolato (vedi Sezione 3.5).

2.2 I dati: fonti autoritative e generazione sintetica

Il dataset combina **cataloghi autoritativi** e **dati sintetici** chiaramente dichiarati come tali:

- *Autoritativi* (fonti ufficiali): comuni italiani con codice catastale e provincia; codici e settori CCNL dall'export CNEL; codici ATECO dalla classificazione ufficiale;
- *Calibrati sui cedolini reali* (codifica propria): tipi voce, tipi contributo, causali di assenza;
- *Sintetici* (dichiarati): livelli e minimi tabellari, maggiorazioni; e l'intero insieme di ditte, dipendenti e cedolini, generato per rispettare il GDPR.

La generazione è affidata a uno script Python (con **Faker** e un pool di anagrafiche riusate per velocità) parametrico e **riproducibile grazie a un seed fisso**: lo stesso comando produce sempre lo stesso dataset e lo esporta in due formati, uno per il caricamento relazionale (**COPY**) e uno per quello documentale (**mongoimport**), così i due sistemi ricevono *gli stessi dati*. La scala è vincolata dal disco delle macchine (Capitolo 4); i volumi effettivi sono in Tabella 2.1. L'esperimento a dati crescenti su MongoDB ha spinto il volume più in alto, fino a 2630 ditte (Capitolo 5).

Tabella 2.1: Volumi del dataset: composizione del dataset base (30 ditte) e intervallo dei punti di scala usati nell’esperimento a dati crescenti (Capitolo 5).

Scala	Ditte	Dipendenti	Cedolini
Base	30	554	6 648
Dati crescenti (Citus)	60–480	$\approx 1\,100$ – $8\,800$	$\approx 13\,000$ – $106\,000$

2.3 Fondamenti teorici

2.3.1 Modello dei dati: relazionale vs documentale

Nel modello **relazionale** i dati sono organizzati in relazioni (tabelle) di tuple con schema rigido e tipato; le relazioni tra entità si esprimono con chiavi esterne e si ricompongono a interrogazione con i *join*, e il DBMS garantisce l’integrità (chiavi, vincoli). Il pregio è l’assenza di ridondanza (ogni fatto in un solo posto) e la flessibilità delle interrogazioni ad-hoc; il costo è che un’entità “logica” come il cedolino completo è spezzata su più tabelle e va ricomposta con *join* [1].

Nel modello **documentale** i dati sono documenti annidati (BSON/JSON) raccolti in collezioni, con schema flessibile; le relazioni si modellano soprattutto con l’**embedding** (annidare i dati correlati nello stesso documento, ottenendo l’aggregato). Il pregio è che l’aggregato che si legge e si scrive insieme sta in un solo documento (nessun *join*, una sola operazione di I/O) e che lo schema evolve senza migrazioni; il costo è la **denormalizzazione** (lo stesso dato può essere duplicato in molti documenti, e aggiornarlo diventa un *mass-update*) e l’assenza di integrità referenziale a carico del DBMS. Nessun modello è universalmente migliore: la scelta dipende dalle operazioni prevalenti [2].

2.3.2 Linguaggi di interrogazione

Il relazionale usa **SQL**, dichiarativo e potente per l’analitica ad-hoc (*join*, **GROUP BY**, funzioni finestra, percentili). MongoDB usa query semplici (**find**) e una **aggregation pipeline** a stadi (**\$match**, **\$group**, **\$unwind**, **\$lookup**); le *join* (**\$lookup**) esistono ma sono meno naturali, e il modello spinge a *embeddare* ciò che serve insieme. I due linguaggi caratterizzano sistemi diversi: i loro throughput non sono direttamente confrontabili, per cui il confronto equo avviene sulle *stesse operazioni* del dominio e a livello di modello.

2.3.3 Scalabilità e sharding

Oltre una certa scala la crescita verticale (macchina più grande) non basta e serve la crescita orizzontale (*scale-out*), che richiede di **partizionare** i dati tra più nodi tramite una chiave di distribuzione. **Citus** partiziona le tabelle per hash di **tenant_id** in un numero fisso di shard sui worker, replica i cataloghi come *reference table* e co-locata tutte le tabelle di un tenant sullo stesso shard; il *coordinator* instrada le query (a un solo shard o a tutti, con *merge*); l’aggiunta di un nodo richiede un **rebalance esplicito** [3]. **MongoDB** partiziona per *shard key* in *chunk* (intervalli di chiave), instradati dal *mongos*; un processo di **balancer**, attivo di default sul config server, redistribuisce automaticamente i chunk, ma solo quando lo sbilanciamento tra shard supera **3 volte la dimensione del range** (default 128 MB $\Rightarrow$ 384 MB) [4]. Questa soglia si rivelerà decisiva nei risultati (Capitolo 5).

2.3.4 Transazioni, concorrenza e consistenza

Entrambi i sistemi offrono transazioni **ACID**. In Citus una transazione intra-tenant è su un solo nodo (economica), mentre una transazione cross-shard usa il *two-phase commit*; entrambi usano il controllo di concorrenza multiversione (MVCC). MongoDB offre transazioni multi-documento ACID (con snapshot isolation), ma il modello a documenti spesso le *evita*: l'inserimento di un aggregato in un unico documento è già atomico di per sé.

2.3.5 Teorema CAP e PACELC

Il teorema **CAP** afferma che, di fronte a un partizionamento di rete (P), un sistema distribuito non può garantire insieme consistenza (C) e disponibilità (A) [5]. Poiché le partizioni accadono, la scelta è tra **CP** (resta consistente, sacrifica la disponibilità della parte irraggiungibile) e AP. **Entrambi i sistemi qui studiati sono CP**: in un guasto privilegiano la consistenza. L'estensione **PACELC** aggiunge che, anche in assenza di partizioni (Else), si sceglie tra latenza (L) e consistenza (C): i due sistemi sono PC/EC, coerentemente con un dominio retributivo in cui non si tollerano cedolini inconsistenti [6].

2.4 Scelta dei due sistemi

PostgreSQL + Citus. Serviva un relazionale *realmente distribuito*: PostgreSQL “liscio” non lo è, mentre i sistemi NewSQL sono database a sé. Citus è invece un'estensione di PostgreSQL con primitive di distribuzione *esplicite* (tabelle distribuite, reference table, co-localizzazione) che si mappano in modo diretto sul discorso di modellazione ed è quindi didattico; offre SQL completo, transazioni distribuite e consistenza forte (CP).

MongoDB. Come controparte NoSQL è stato scelto MongoDB (documentale) invece di Cassandra (colonnare, AP). Cassandra darebbe un contrasto CAP più estremo, ma il progetto richiede esplicitamente le *transazioni*: le transazioni ACID general-purpose di Cassandra non sono ancora nelle versioni stabili, mentre MongoDB le ha stabili da anni. Inoltre il modello a documenti si adatta bene all'aggregato “cedolino” e rende quasi gratuita l'evoluzione di schema. Si ottiene così un confronto CP↔CP equilibrato ma tra *modelli di dati opposti*.

Tipo di sharding: row-based. Citus offre due modalità di distribuzione (per riga su una colonna, o per schema). Si è scelto il **row-based** (partizionamento per `tenant_id`) perché è l'unico simmetrico a MongoDB, che distribuisce per shard key su una collezione: solo così il confronto è bilanciato, e l'analitica cross-tenant parallelizza nativamente sugli shard.

3 Progettazione delle basi di dati

Lo stesso dominio è stato modellato nei due paradigmi: uno schema concettuale unico (entità-relazione) è stato tradotto in uno schema logico *relazionale distribuito* per Citus e in uno schema *documentale* per MongoDB. Il confronto tra le due traduzioni è già di per sé un risultato.

3.1 Schema concettuale (ER)

Il modello concettuale adotta la notazione entità-relazione di Atzeni [1]. Il dominio completo conta circa venticinque entità; per leggibilità se ne dà una vista *semplificata* con le entità principali (Figura 3.1) e la vista *completa* (Figura 3.2). Le scelte di traduzione notevoli: la generalizzazione DITTA (persona giuridica/fisica) è ristrutturata in relazioni 1:1 con discriminante; le associazioni N:N (ditta-ATECO, ditta-CCNL) diventano tabelle ponte; le entità deboli (voci, ratei, timbrature) ereditano la chiave del padre.

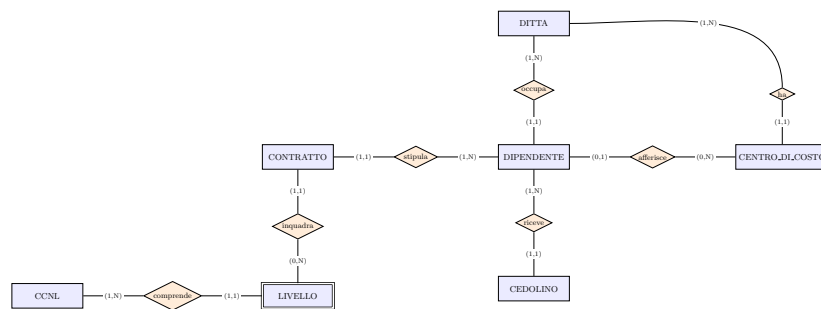


Figura 3.1: Schema ER semplificato: le entità principali del dominio HR/retributivo.

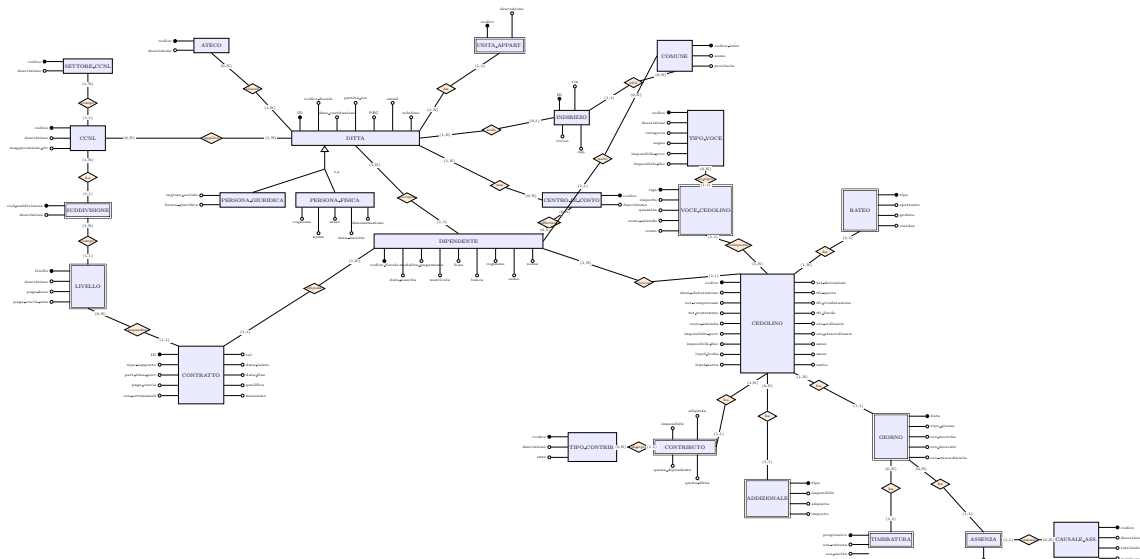


Figura 3.2: Schema ER completo (≈ 25 entità).

3.2 Schema logico relazionale (Citus)

La traduzione al relazionale distribuito distingue due categorie di tabelle (Tabella 3.1): **reference table**, cioè i cataloghi condivisi e quasi statici, replicati su ogni nodo per consentire join locali; e **tabelle distribuite**, cioè tutto ciò che appartiene al tenant, partizionato per hash di `tenant_id`. Le tabelle distribuite stanno in un *unico gruppo di co-locazione*: un intero tenant vive su un solo shard, quindi join e transazioni intra-tenant sono *single-node* (nessun 2PC né traffico di rete). La distribuzione si dichiara con poche primitive:

```
1 -- Cataloghi condivisi: reference table (replicate su ogni nodo)
2 SELECT create_reference_table('tipo_voce');
3 -- Tabelle del tenant: distribuite per hash di tenant_id, co-locate tra loro
4 SELECT create_distributed_table('dipendente', 'tenant_id');
5 SELECT create_distributed_table('cedolino', 'tenant_id', colocate_with => 'dipendente');
6 SELECT create_distributed_table('voce_cedolino', 'tenant_id', colocate_with => 'dipendente');
```

Listing 3.1: Dichiarazione della distribuzione in Citus.

Tabella 3.1: Ripartizione delle tabelle in Citus tra reference (replicate) e distribuite (per `tenant_id`).

Tipo	Tabelle
Reference (replicate su ogni nodo)	comune, ateco, settore_ccnl, ccnl, suddivisione, livello, tipo_voce, tipo_contributo, causale_assenza
Distribuite (hash di <code>tenant_id</code> , co-locate)	ditta, persona_giuridica, persona_fisica, indirizzo, centro_costo, dipendente, contratto, cedolino, voce_cedolino, rateo, contributo, addizionale, timbratura, giorno, assenza, ...

Un vincolo di Citus è che ogni chiave primaria o **UNIQUE** di una tabella distribuita deve includere la colonna di distribuzione: la matricola è quindi unica *dentro il tenant*, e il codice fiscale non è globalmente unico (la stessa persona può lavorare per più ditte). La co-locazione è illustrata in Figura 3.3.

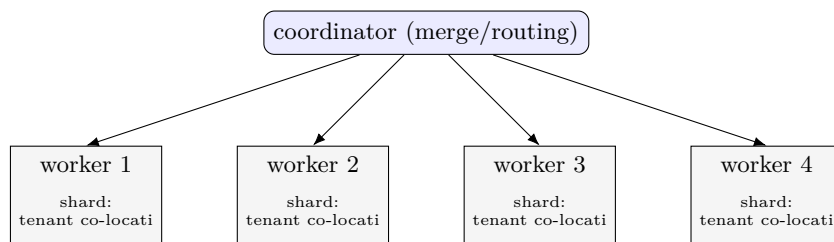


Figura 3.3: Co-locazione in Citus: ogni tenant (con tutte le sue tabelle) risiede su un solo shard/-worker; il coordinator instrada le query single-shard a un worker e fa il merge di quelle cross-shard.

3.3 Schema documentale (MongoDB)

Nel modello documentale il cedolino diventa **un unico documento** che *embedda* tutti i suoi annidati (Elenco 3.2); le collezioni principali sono tre, tutte shardate per `tenant_id` (hashed), mentre i cataloghi restano collezioni non shardate (Tabella 3.2).


```

1 db.cedolini.insertOne({
2   tenant_id: 1, cedolino_id: 42, dipendente_id: 7, anno: 2026, mese: 6,
3   dipendente: { sesso: "M", livello: "3", ccnl: "A014", cdc: 1 },
4   totali: { competenze: 2500, trattenute: 700, netto: 1800, costo_azienza: 3300 },
5   voci: [ { riga: 1, tipo: "0201", importo: 2000 }, { riga: 2, tipo: "0301", importo: 500 }
6   ],
7   ratei: [ { tipo: "FERIE", residuo: 2.16 } ],
8   contributi: [ { tipo: "INPS_FPLD", quota_dipendente: 229.75, quota_ditta: 595.25 } ],
9   addizionali: [ { tipo: "REGIONALE", importo: 39.27 } ]
10 });

```

Listing 3.2: Documento “cedolino”: l’aggregato completo in un solo documento (embedding).

Tabella 3.2: Collezioni MongoDB e chiave di sharding.

Collezione	Contenuto	Shard key
ditte	anagrafica della ditta (persona giur./fisica, indirizzi, centri di costo embedded)	tenant_id (hashed)
dipendenti	dipendente con i <i>contratti</i> embedded	tenant_id (hashed)
cedolini	cedolino completo (voci, ratei, contributi, addizionali embedded)	tenant_id (hashed)
cataloghi	comuni, ateco, tipi voce/contributo, causali	non shardate

La differenza cruciale è nell’**intestazione del cedolino**: nel relazionale l’anagrafica non si duplica nel cedolino (si naviga via relazioni), mentre il documentale ne embedda uno *snapshot*. È esattamente il punto in cui i due modelli divergono e che verrà misurato.

3.4 Quando il modello modella bene

La suite di test include coppie di operazioni deliberatamente scelte per esporre i punti di forza e di debolezza dei due modelli:

- **Pro-documentale:** leggere e scrivere il cedolino *completo*. In Mongo è un `findOne/insertOne` su un documento; in Citus è un assemblaggio (o una transazione) su sette tabelle co-locate.
- **Pro-relazionale:** modificare un *dato di riferimento* (la categoria di un tipo voce). In Citus è **una riga** su una reference table; in Mongo, con il dato denormalizzato nei cedolini, è un `updateMany` di centinaia di documenti.

I risultati di queste coppie sono in Capitolo 5.

3.5 Evoluzione dello schema

Il requisito di gestire l’evoluzione dello schema in corso d’opera (es. l’aggiunta di turni e ferie a un sistema già popolato) evidenzia un’altra divergenza, qui trattata a livello di principio. In **Citus/PostgreSQL** un cambiamento strutturale richiede un `ALTER TABLE` (o nuove tabelle distribuite) e un *backfill* dei dati: transazionale e verificato dai vincoli, ma con possibili lock e da propagare a tutti gli shard. In **MongoDB**, grazie allo schema flessibile, l’aggiunta di campi o array è quasi gratuita: documenti vecchi e nuovi coesistono e l’assenza dei nuovi campi è gestita dall’applicazione (schema-on-read). Il documentale è più agile sulle estensioni, il relazionale offre in cambio garanzie di integrità durante la migrazione.

4 Metodologia sperimentale

Le misure sono state condotte su un cluster reale in cloud, con la stessa topologia e lo stesso dataset per i due sistemi, e con strumenti che rilevano l'impatto *di ciascun nodo*, non solo il tempo visto dal client. Questo capitolo descrive infrastruttura, deploy, suite di query e strumenti di misura.

4.1 Infrastruttura

Il cluster è composto da cinque macchine virtuali Azure nella stessa subnet privata (Tabella 4.1). La scelta di macchine *separate* è deliberata: con client e nodi DB sulla stessa macchina non si vedrebbe l'impatto della rete e il disco farebbe da collo di bottiglia. La macchina più capace (worker-1, 16 vCPU) fa da **coordinator** in Citus e ospita *config server* e *mongos* in MongoDB: è il punto di merge/instradamento e resta fisso durante lo scaling, così un coordinator potente non maschera il beneficio di aggiungere worker. I quattro nodi-dati (VM-1... VM-4, 4 vCPU ciascuno) sono **simmetrici**, così lo scaling è equo. Il traffico interno viaggia sugli IP privati (container in `network_mode: host`, niente NAT); le porte DB non sono esposte a internet.

Tabella 4.1: Topologia del cluster: 5 VM Azure, stessa subnet privata.

Ruolo	VM (IP priv.)	vCPU	RAM	Citus	MongoDB
Coordinator	worker-1 (10.0.1.8)	16	32 GB	coordinator	config server + mongos
Data node	VM-1 (10.0.1.4)	4	16 GB	worker	shard
Data node	VM-2 (10.0.1.5)	4	16 GB	worker	shard
Data node	VM-3 (10.0.1.7)	4	16 GB	worker	shard
Data node	VM-4 (10.0.1.6)	4	16 GB	worker	shard

4.2 Deploy e gestione dei nodi

Ogni ruolo è impacchettato in un `docker-compose` dedicato (versioni pinnate: Citus 14.1-alpine, MongoDB 8.3), con autenticazione completa (SCRAM e `.pgpass` per Citus; keyfile e utente admin per Mongo). Il deploy porta su i servizi “nudi”, *non* registrati: l'aggiunta dei nodi al cluster avviene **a mano** (`citus_add_node` / `sh.addShard`), per avere il controllo necessario agli esperimenti di scaling. Qui emerge una differenza operativa importante: in Citus, dopo aver aggiunto un worker, la redistribuzione degli shard è **esplicita** (`rebalance_table_shards`, sincrona); in MongoDB è affidata al **balancer automatico**, asincrono e soggetto a soglia. Questa differenza avrà un peso decisivo sui risultati (Capitolo 5).

```
1 # Deploy dei container (versioni pinnate) su tutte le VM
2 bash infra/prod/deploy/deploy-citus.sh # Citus: coordinator + worker
3 bash infra/prod/deploy/deploy-mongo.sh # Mongo: config server + mongos + shard
4 # Citus: schema (reference + distribuite) e dati
5 docker exec -i citus psql -U postgres -d archdata < schema/relational/01_reference.sql
6 docker exec -i citus psql -U postgres -d archdata < schema/relational/02_distributed.sql
7 python3 data/genera.py --tenant-count 30 --dip-min 10 --dip-max 25 --seed 42 --out generato
8 CITUS_CONTAINER=citus bash scripts/carica-citus.sh generato/citus
```

```
9 # MongoDB: cluster a 4 shard pulito + sharding collezioni + caricamento
10 MONGO_PASSWORD=... bash setup-mongo-4shard.sh
```

Listing 4.1: Deploy dei cluster, inizializzazione dello schema e caricamento del dataset (riproducibile).

4.3 Suite di query speculare

Il confronto equo si basa su una suite di query **identica nella semantica** sui due sistemi: **13 letture** (L01...L13) e **7 scritture** (inserimenti, aggiornamenti, cancellazione), una per file, con nomi speculari tra la versione SQL (`.sql`) e quella MongoDB (`.js`). Le letture vanno dalla query puntuale single-shard (il cedolino di un dipendente) alle analitiche cross-tenant con percentili e join ai cataloghi (il gender pay gap mediano per livello). Ogni query è parametrica.

Un accorgimento importante per un test distribuito realistico: nelle **letture** il carico **randomizza** i parametri (tenant, dipendente, mese) a ogni iterazione, così le richieste si distribuiscono sui vari shard e non colpiscono sempre lo stesso documento; nelle **scritture** il tenant è invece *fisso*, per misurare l'effetto della co-locazione (la scrittura di un tenant colpisce un solo nodo).

4.4 Strumenti di misura per-nodo

Per ogni query si misura la latenza e il throughput sotto *carico sostenuto*, ma soprattutto *quanto lavora ciascun nodo*. Gli strumenti sono nativi di ciascun sistema:

- **Citus:** il carico è generato con `pgbench` (connessione persistente); i contatori per-nodo (righe elaborate, tempo di esecuzione, letture da cache/disco, WAL) si leggono da `pg_stat_statements` propagato a tutti i nodi con `run_command_on_all_nodes()`, con azzeramento e delta attorno a ogni misura.
- **MongoDB:** il carico è un ciclo `mongosh`; i contatori per-shard vengono da `serverStatus().opcounters`, l'uso di CPU/RAM da `docker stats` campionato su ogni nodo.
- **Sistema operativo:** CPU, RAM e I/O di disco per nodo si campionano da `/proc` durante ogni run, raccogliendo tutto in CSV strutturati (fonte dei grafici e delle tabelle del Capitolo 5).

Due note metodologiche. Le query si eseguono *una alla volta*, senza accesso concorrente, così l'impatto è attribuibile alla singola query e leggibile su ogni nodo. E la relazione **cache/disco**: a scala piccola il dataset entra tutto in RAM, quindi le letture non toccano il disco (`blk_read=0`); le letture fisiche da disco compaiono solo quando i dati superano la RAM, cioè nell'esperimento a dati crescenti, ed è lì che il disco diventa il collo di bottiglia. Per questo lo scaling per nodi si misura a freddo (cache svuotata) e quello a dati crescenti a caldo (per vedere la capacità massima e il punto in cui la cache cede).

5 Risultati e analisi

Questo capitolo raccoglie le misure dei test e le analizza. Salvo diverso avviso i numeri sono medie su tre esecuzioni a freddo, ovvero la cache viene svuotata prima di ogni esecuzione, per lo scaling per nodi, e misure a caldo per lo scaling sui dati. Il dataset di partenza è delle stesse dimensioni su entrambi i sistemi (30 ditte, 554 dipendenti), generato con lo script dedicato.

5.1 Baseline in lettura e i due regimi

Su Citus a un nodo, sotto carico sostenuto, emergono due regimi netti. Le query **single-shard** (cedolino singolo, cedolini di una ditta, riepiloghi per un tenant) stanno a **1–3 ms** con throughput fino a ~ 680 op/s: colpiscono un solo shard e il coordinator ha solo un overhead fisso di instradamento. Le analitiche **cross-shard** (percentili, aggregazioni cross-tenant, spesa per categoria) stanno a **55–85 ms** con 13–18 op/s: ogni worker satura la CPU per lo scan (100–190 %, multi-core) e il coordinator (~ 40 %) fa il merge. Questa distinzione tra i due regimi si ritrova in tutti gli esperimenti seguenti.

5.2 Scalabilità per numero di nodi (Citus)

A parità di dati si passa da 1 a 4 nodi, ridistribuendo gli shard a ogni passo. Le query cross-shard scalano in modo **quasi lineare** (Tabella 5.1 e fig. 5.1): con 4 nodi lo speedup è **3.8–4.6×**. Le single-shard restano invece piatte: colpiscono un solo shard su un solo nodo, quindi aggiungere worker non le tocca (ma sono già velocissime). Il lavoro si distribuisce: la CPU e il tempo di esecuzione *per worker* calano proporzionalmente (per L04, la CPU del worker passa da 142 % a 24 % e le righe per worker si dimezzano a ogni raddoppio).

Tabella 5.1: Latenza (ms) al crescere dei nodi e speedup 1→4. Cross-shard vs single-shard.

Query	1 nodo	2 nodi	3 nodi	4 nodi	speedup
L04 gender gap mediano	71.9	45.0	23.2	18.3	3.9×
L05 top-N retribuzioni	65.8	33.7	19.4	14.9	4.4×
L06 assenteismo/mese	61.4	31.2	19.9	15.4	4.0×
L08 trend costo lavoro	56.1	24.0	15.9	12.1	4.6×
L11 spesa/categoria	75.6	52.7	25.3	20.1	3.8×
L12 ferie residue	75.4	59.7	25.2	19.5	3.9×
L01 cedolino singolo (<i>single-shard</i>)	1.5	1.5	1.5	1.5	1.0×

Il dettaglio completo di *tutte* le metriche raccolte (latenza, throughput, CPU per worker, tempo di esecuzione e righe elaborate per worker) per ognuna delle 13 query è in Tabella 5.2: mostra esplicitamente come, all’aumentare dei nodi, il lavoro *per worker* (CPU, **exec**, righe) si riduca, mentre l’overhead sul coordinator resta pressoché costante.

Tabella 5.2: Scalabilità per numero di nodi (Citius): tutte le metriche raccolte, per le 13 query, a 1–4 nodi. Le metriche per-worker sono la media sui worker attivi.

Query	Parametro	1 nodo	2 nodi	3 nodi	4 nodi
L01 cedolino singolo	latenza (ms)	1.5	1.5	1.5	1.5
	throughput (op/s)	680	666	669	688
	CPU worker (%)	11	6	4	3
	exec worker (ms)	138	63	47	34
	righe/worker	6797	3331	2230	1718
L02 cedolini ditta/periodo	latenza (ms)	1.5	1.6	1.6	1.5
	throughput (op/s)	669	636	638	654
	CPU worker (%)	15	7	5	4
	exec worker (ms)	171	82	56	43
	righe/worker	6699	3182	2127	1636
L03 costo per centro	latenza (ms)	2.3	2.3	2.3	2.3
	throughput (op/s)	438	431	437	433
	CPU worker (%)	27	14	9	7
	exec worker (ms)	646	315	213	161
	righe/worker	210192	103456	69909	51988
L04 gender gap mediano	latenza (ms)	71.9	45.0	23.2	18.3
	throughput (op/s)	14	23	43	55
	CPU worker (%)	142	57	26	24
	exec worker (ms)	531	335	367	347
	righe/worker	77375	63064	79899	75667
L05 top-N retribuzioni	latenza (ms)	65.8	33.7	19.4	14.9
	throughput (op/s)	15	31	52	67
	CPU worker (%)	129	33	20	20
	exec worker (ms)	327	237	244	266
	righe/worker	54840	55080	61960	60240
L06 assenteismo per mese	latenza (ms)	61.4	31.2	19.9	15.4
	throughput (op/s)	16	32	50	65
	CPU worker (%)	122	30	21	17
	exec worker (ms)	1700	1225	1224	923
	righe/worker	245000	242750	251667	187125
L07 straordinari per centro	latenza (ms)	3.1	3.1	3.1	3.2
	throughput (op/s)	321	322	326	317
	CPU worker (%)	41	21	14	11
	exec worker (ms)	1631	889	603	441
	righe/worker	12843	6435	4343	3165
L08 trend costo lavoro	latenza (ms)	56.1	24.0	15.9	12.1
	throughput (op/s)	18	42	63	82
	CPU worker (%)	106	14	12	12
	exec worker (ms)	466	368	360	351
	righe/worker	42880	50080	50267	49500
L09 organico per livello	latenza (ms)	57.7	24.5	17.0	12.8
	throughput (op/s)	17	41	59	78
	CPU worker (%)	106	15	14	13
	exec worker (ms)	186	134	127	125
	righe/worker	51852	60991	58640	58234
L10 cedolino completo	latenza (ms)	82.7	63.3	27.9	21.7
	throughput (op/s)	12	16	36	46
	CPU worker (%)	189	91	33	31
	exec worker (ms)	169	97	116	112
	righe/worker	121	79	120	115
L11 spesa per categoria	latenza (ms)	75.6	52.7	25.3	20.1

Query	Parametro	1 nodo	2 nodi	3 nodi	4 nodi
	throughput (op/s)	13	19	40	50
	CPU worker (%)	154	68	27	25
	exec worker (ms)	1464	535	524	543
	righe/worker	73497	52907	73066	69158
	latenza (ms)	75.4	59.7	25.2	19.5
L12 ferie residue	throughput (op/s)	13	17	40	51
	CPU worker (%)	165	82	32	30
	exec worker (ms)	909	439	545	528
	righe/worker	3990	2520	3963	3842
	latenza (ms)	3.4	3.5	3.5	3.4
L13 riepilogo assenze	throughput (op/s)	291	290	289	291
	CPU worker (%)	38	19	12	10
	exec worker (ms)	923	453	305	230
	righe/worker	29320	14487	9628	7281
	latenza (ms)	3.4	3.5	3.5	3.4

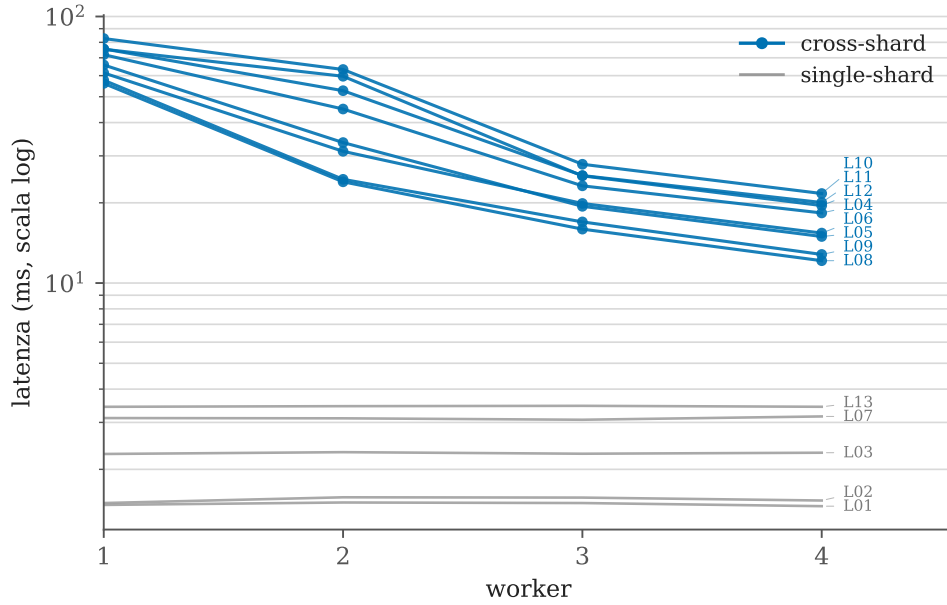


Figura 5.1: Citus, scaling per nodi (tutte le 13 letture, scala log; codice query in coda a ogni linea): le cross-shard calano nettamente al crescere dei nodi, le single-shard restano piatte.

```

1 bash citus-add-node.sh 10.0.1.5 && bash citus-rebalance.sh # ad ogni passo di scaling
2 for i in 1 2 3; do bash azzera-cache-citus.sh; \
3   bash esegui-suite-citus.sh 10 queries/relational/letture/*.sql; done
4 bash media-citus.sh

```

Listing 5.1: Esecuzione: aggiunta di un worker e rebalance, poi 3 run a freddo delle 13 letture e media (ripetuto a 1, 2, 3, 4 nodi).

5.3 Scalabilità per volume di dati (Citus)

A numero di nodi fisso (4) si aumenta il volume dei dati. Le cross-shard **degradano in modo super-lineare** (Tabella 5.3 e fig. 5.2): da 30 a 480 ditte L11 passa da 20 a 114 ms (5.7×), con un salto netto tra 240 e 480 ditte. Le single-shard restano **piatte** (~1.5 ms a ogni scala): indicizzate e mirate a un tenant, sono insensibili al volume globale, che è il vantaggio

del multitenant co-locato. Le letture da disco (**blk_read**) restano nulle: a questa scala i dati stanno ancora in RAM, quindi il degrado è *CPU-bound* (più righe da scandire in cache). L'esperimento si è fermato a 480 ditte perché a quel punto la tendenza era già chiara e il carico risultava distribuito su tutti i nodi: proseguire non avrebbe aggiunto informazione.

Tabella 5.3: Latenza (ms) al crescere dei dati, a 4 nodi fissi.

Ditte	L01 single-shard	L04 gender gap	L06 assenteismo	L11 spesa
30 (base)	1.5	18.3	15.4	20.1
60	1.5	20.4	17.5	24.6
120	1.5	24.0	25.1	34.9
240	1.4	30.9	51.2	66.0
480	1.6	68.3	73.1	114.1

Il dettaglio completo di tutte le metriche per le 13 query, a ogni scala, è in Tabella 5.4: si vede come il tempo di esecuzione e le righe elaborate per worker crescano col volume, mentre le query single-shard restano invariate.

Tabella 5.4: Scalabilità per volume di dati (Citius, 4 nodi fissi): tutte le metriche per le 13 query, da 30 a 480 ditte.

Query	Parametro	30	60	120	240	480
L01 cedolino singolo	latenza (ms)	1.5	1.5	1.5	1.4	1.6
	throughput (op/s)	688	659	655	705	643
	CPU worker (%)	3	6	6	5	4
	exec worker (ms)	34	36	36	38	34
	righe/worker	1718	1647	1636	1762	1607
L02 cedolini ditta/periodo	latenza (ms)	1.5	1.5	1.5	1.6	1.5
	throughput (op/s)	654	659	674	635	682
	CPU worker (%)	4	5	4	4	4
	exec worker (ms)	43	44	45	33	36
	righe/worker	1636	1648	1684	1588	1706
L03 costo per centro	latenza (ms)	2.3	2.3	2.4	2.4	2.4
	throughput (op/s)	433	438	424	414	423
	CPU worker (%)	7	7	7	8	8
	exec worker (ms)	161	164	171	175	185
	righe/worker	51988	52548	50820	49680	50748
L04 gender gap mediano	latenza (ms)	18.3	20.4	24.0	30.9	68.3
	throughput (op/s)	55	49	42	32	15
	CPU worker (%)	24	26	26	28	38
	exec worker (ms)	347	623	947	1430	1325
	righe/worker	75667	164762	283664	454734	411049
L05 top-N retribuzioni	latenza (ms)	14.9	14.8	15.8	15.7	17.4
	throughput (op/s)	67	68	63	64	58
	CPU worker (%)	20	22	23	27	30
	exec worker (ms)	266	423	587	953	1475
	righe/worker	60240	89401	98425	102080	92320
L06 assenteismo per mese	latenza (ms)	15.4	17.5	25.1	51.2	73.1
	throughput (op/s)	65	57	40	20	14
	CPU worker (%)	17	32	39	68	91
	exec worker (ms)	923	2445	3337	3655	5274
	righe/worker	187125	326326	262342	134554	94256
L07 straordinari per centro	latenza (ms)	3.2	3.2	3.2	3.2	3.3
	throughput (op/s)					
	CPU worker (%)					
	exec worker (ms)					
	righe/worker					

Query	Parametro	30	60	120	240	480
	throughput (op/s)	317	317	309	312	307
	CPU worker (%)	11	11	11	11	12
	exec worker (ms)	441	478	474	483	480
	righe/worker	3165	3173	3093	3122	3068
L08 trend costo lavoro	latenza (ms)	12.1	12.6	13.9	14.9	17.8
	throughput (op/s)	82	79	72	67	56
	CPU worker (%)	12	14	18	27	39
	exec worker (ms)	351	673	1148	2091	3460
	righe/worker	49500	68904	66960	64608	54144
L09 organico per livello	latenza (ms)	12.8	13.5	14.6	16.8	20.2
	throughput (op/s)	78	74	69	60	49
	CPU worker (%)	13	13	14	15	17
	exec worker (ms)	125	210	336	546	881
	righe/worker	58234	104710	185220	321186	535095
L10 cedolino completo	latenza (ms)	21.7	21.7	21.8	21.3	21.7
	throughput (op/s)	46	46	46	47	46
	CPU worker (%)	31	32	31	31	30
	exec worker (ms)	112	115	115	119	116
	righe/worker	115	115	115	118	115
L11 spesa per categoria	latenza (ms)	20.1	24.6	34.9	66.0	114.1
	throughput (op/s)	50	41	29	15	9
	CPU worker (%)	25	27	29	42	52
	exec worker (ms)	543	1102	1292	1404	1718
	righe/worker	69158	136854	195912	213332	246070
L12 ferie residue	latenza (ms)	19.5	20.4	23.5	28.3	49.6
	throughput (op/s)	51	49	43	35	20
	CPU worker (%)	30	35	38	44	65
	exec worker (ms)	528	1007	1576	2569	3100
	righe/worker	3842	7365	12780	21240	24240
L13 riepilogo assenze	latenza (ms)	3.4	3.6	3.6	3.6	3.5
	throughput (op/s)	291	279	276	280	289
	CPU worker (%)	10	10	10	10	10
	exec worker (ms)	230	233	224	228	193
	righe/worker	7281	6978	6895	7010	7220

```
1 bash scala-dati-citus.sh 31 30 60ditte # +30 tenant -> 60 ditte; poi 61 60 120ditte; ...
```

Listing 5.2: Esecuzione: a 4 nodi fissi, un lotto per volta genera nuovi tenant, carica e misura a caldo (ripetuto fino a 480 ditte).

5.4 Scritture distribuite (Citus)

Le scritture, misurate a 4 nodi (Tabella 5.5), mostrano l'effetto della **co-localizzazione**: **cinque scritture su sei colpiscono un solo nodo** (quello dello shard del tenant), mentre gli altri tre worker restano inattivi. La scrittura di un singolo tenant *non si distribuisce* aggiungendo nodi; il throughput aggregato scala solo distribuendo scritture su tenant diversi. Il costo cresce con le righe/tabelle toccate e il WAL prodotto: dalla semplice **UPDATE** (U02, 605 op/s) alla **transazione** multi-tabella I03 (17 op/s, ~1 MB di WAL) c'è un fattore ~35×. L'unica scrittura che tocca **tutti i nodi** è U03, l'aggiornamento di un dato di riferimento su una reference table (replicata ovunque): una riga logica, ma applicata su ogni nodo.

```
1 bash esegui-scritture-citus.sh 300 queries/relational/scritture/*.sql
```

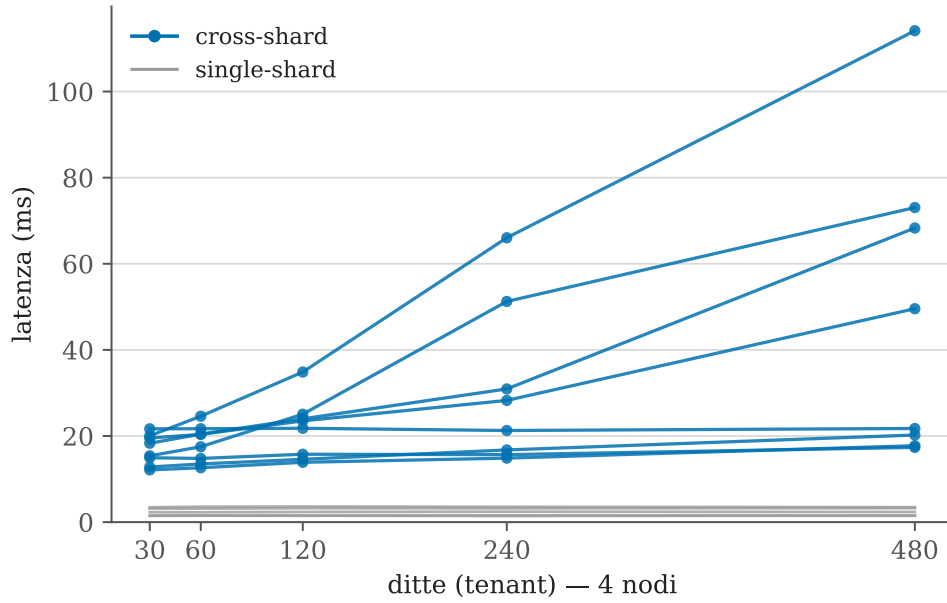



Figura 5.2: Citus, scaling per dati (tutte le 13 letture, 4 nodi fissi): degrado super-lineare delle cross-shard; single-shard insensibili al volume.

Tabella 5.5: Scritture a 4 nodi (N=300 operazioni per query).

Op.	Descrizione	lat (ms/op)	tps	dove scrive
U02	licenziamento (update 1 riga)	1.7	605	solo 1 nodo
U01	coordinate bancarie	3.7	273	solo 1 nodo
U04	update concorrente (riga cond.)	3.8	266	solo 1 nodo
I01	inserimento timbratura	10.6	94	solo 1 nodo
U03	modifica dato di riferimento	21.1	47	tutti i nodi
I03	elaborazione cedolino (transazione)	57.4	17	solo 1 nodo

Listing 5.3: Esecuzione delle scritture (Citus): N=300 ripetizioni per query, tenant fisso.

5.5 Comportamento in presenza di guasti (Citus)

Il comportamento **CP** è stato messo alla prova eseguendo una query cross-shard (L04) in un ciclo mentre un worker scelto a caso viene spento e poi riacceso (11 prove). Il risultato è netto e coerente: spegnendo *qualsiasi* worker, la query fallisce per **tutta** la finestra di down (15s), perché ha bisogno degli shard di ogni nodo, quindi manca sempre qualcosa e **non restituisce risultati parziali** (le righe crollano da N a 0). L'error-rate medio è ~51 %, il recupero è **automatico e rapido** (prima risposta ~0.5s dopo il riavvio del container). I quattro worker danno lo stesso error-rate: ognuno è essenziale, nessuno ridondante (copia singola, `shard_replication_factor=1`). È la consistenza scelta al posto della disponibilità.

```
1 bash test-guasto-citus.sh queries/relational/letture/L04_gender_gap_mediano.sql random 10 15
45
```

Listing 5.4: Esecuzione del test di guasto (Citus): L04 in loop mentre un worker a caso viene spento (10–25 s) e riacceso, su 45 s totali.

5.6 MongoDB

5.6.1 Scalabilità per shard: il balancer non distribuisce a scala piccola

Sul cluster documentale il regime “più shard, stessi dati” ha rivelato un comportamento che è esso stesso il risultato più interessante. Con il dataset base (*cedolini* ≈ 42 MB), il balancer di MongoDB **non distribuisce**: con la dimensione di range di default (128 MB) la soglia di migrazione è 384 MB, mai raggiunta, quindi resta un solo chunk su un solo shard. Anche forzando le condizioni, con il range ridotto a 4 MB (soglia 12 MB) e uno *split manuale* in 10 chunk, il balancer, pur attivo (`mode: full`, 888 round eseguiti), **non ha migrato nulla**. Di conseguenza il secondo shard resta inattivo (0.2% CPU) e le prestazioni a 1 e 2 shard sono **identiche** (Tabella 5.6): aggiungere shard non porta alcun beneficio. È l’opposto di Citus, dove il rebalance esplicito distribuisce sempre. Il balancer distribuisce solo a *volume grande*: già a ≈ 230 ditte (*cedolini* > 300 MB) inizia a spargere i chunk sugli shard, e il comportamento dipende dunque dalla scala.

Tabella 5.6: MongoDB, latenza (ms) a 1 e 2 shard: identica, perché il 2° shard resta inattivo.

Query	1 shard	2 shard
L04 gender gap (aggregazione)	7.2	7.2
L11 spesa per categoria	12.4	12.5
L01 cedolino singolo (mirata)	2.3	2.3

```
1 export MONGO_PASSWORD=...
2 bash scala-nodi-mongo.sh 4 # dal numero di shard attuale fino a 4
```

Listing 5.5: Esecuzione dello scaling per shard (MongoDB): aggiunta automatica degli shard e 3 run a freddo delle letture per passo.

5.6.2 Scalabilità per volume di dati (MongoDB)

Applicando lo *stesso* protocollo delle controparti Citus (identica suite, misura a caldo, carico randomizzato, contatori per shard), a numero di shard fisso (4) si aumenta il volume dei dati. Qui si completa il quadro emerso con lo scaling per shard: mentre a scala piccola il balancer non distribuiva, **a volume grande distribuisce davvero**. A ogni scala i *cedolini* risultano spartiti su **tutti e 4 gli shard**, attivi al 48–65% di CPU. Le query cross-shard degradano super-linearmente (Tabella 5.7): L04 26→185 ms, L11 34→308 ms; le più pesanti, L06 e L12, crescono a dismisura (302→2834 ms e 476→4786 ms) perché aggregano tutti i *cedolini* scorrendone gli array annidati: a scala grande il costo dell’embedding sull’intera collezione diventa dominante. Le query mirate restano piatte (L01 ~ 2.6 ms).

Tabella 5.7: MongoDB, scalabilità per dati (4 shard fissi): latenza (ms) al crescere del volume.

Ditte	L01	L04	L11	L06	L12
230	2.6	26.0	34.2	301.6	476.2
630	2.6	61.6	80.0	729.1	1174.1
1430	2.7	122.2	178.0	1684.9	2759.5
2630	2.7	185.2	308.0	2833.8	4786.3

Il dettaglio completo per tutte le 13 query e tutte le metriche (latenza, throughput, CPU media per shard, I/O di disco letto e scritto) è in Tabella 5.8. Due osservazioni: la CPU

cresce su *tutti* e quattro gli shard (a conferma che il balancer, a questo volume, ha distribuito), e le letture da disco restano nulle anche a 2630 ditte: il working set è ancora servito dalla cache WiredTiger, quindi il degrado è dominato dal numero di documenti esaminati, non dall'I/O.

Tabella 5.8: Scalabilità per volume di dati (MongoDB, 4 shard fissi): tutte le metriche raccolte per le 13 query, da 230 a 2630 ditte. Le metriche per-shard sono la media/somma sui 4 shard.

Query	Parametro	230	630	1430	2630
L01 cedolino singolo	latenza (ms)	2.6	2.6	2.7	2.7
	throughput (op/s)	385	383	367	374
	CPU media/shard (%)	6	5	5	6
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	139.5	2.5	3.7	748.9
L02 cedolini ditta/periodo	latenza (ms)	3.0	3.1	3.2	3.1
	throughput (op/s)	329	325	316	321
	CPU media/shard (%)	6	6	6	6
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	5.6	153.9	112.1	4.1
L03 costo per centro	latenza (ms)	3.4	3.4	3.5	3.5
	throughput (op/s)	294	291	282	289
	CPU media/shard (%)	6	7	6	6
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	4.5	4.8	2.6	5.8
L04 gender gap mediano	latenza (ms)	26.0	61.6	122.2	185.2
	throughput (op/s)	38	16	8	5
	CPU media/shard (%)	14	15	18	22
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	5.4	6.0	5.2	2.9
L05 top-N retribuzioni	latenza (ms)	7.9	14.0	26.8	44.4
	throughput (op/s)	127	72	37	22
	CPU media/shard (%)	31	43	54	58
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	3.7	8.8	7.9	7.4
L06 assenteismo per mese	latenza (ms)	301.6	729.1	1684.9	2833.8
	throughput (op/s)	3	1	1	0
	CPU media/shard (%)	59	66	70	75
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	7.2	6.6	5.1	5.1
L07 straordinari per centro	latenza (ms)	3.6	3.7	3.8	3.8
	throughput (op/s)	274	270	261	265
	CPU media/shard (%)	8	8	8	8
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	3.3	3.9	4.4	2.6
L08 trend costo lavoro	latenza (ms)	29.0	68.8	149.4	255.5
	throughput (op/s)	34	14	7	4
	CPU media/shard (%)	49	58	64	69
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	9.9	4.6	5.4	10.8
L09 organico per livello	latenza (ms)	19.7	44.6	83.6	125.1
	throughput (op/s)	51	22	12	8
	CPU media/shard (%)	14	16	20	23
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	2.7	10.3	12.5	3.6

Query	Parametro	230	630	1430	2630
L10 cedolino completo	latenza (ms)	2.4	2.5	2.5	2.5
	throughput (op/s)	413	406	396	407
	CPU media/shard (%)	4	3	3	3
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	8.2	3.3	5.0	6.0
L11 spesa per categoria	latenza (ms)	34.2	80.0	178.0	308.0
	throughput (op/s)	29	12	6	3
	CPU media/shard (%)	48	56	61	65
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	3.1	4.8	4.3	3.5
L12 ferie residue	latenza (ms)	476.2	1174.1	2759.5	4786.3
	throughput (op/s)	2	1	0	0
	CPU media/shard (%)	57	62	68	89
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	12.5	5.7	6.1	12.3
L13 riepilogo assenze	latenza (ms)	33.8	34.4	42.8	38.6
	throughput (op/s)	30	29	23	26
	CPU media/shard (%)	17	17	17	21
	disco letto (MB)	0.0	0.0	0.0	0.0
	disco scritto (MB)	3.1	10.4	9.2	2.9

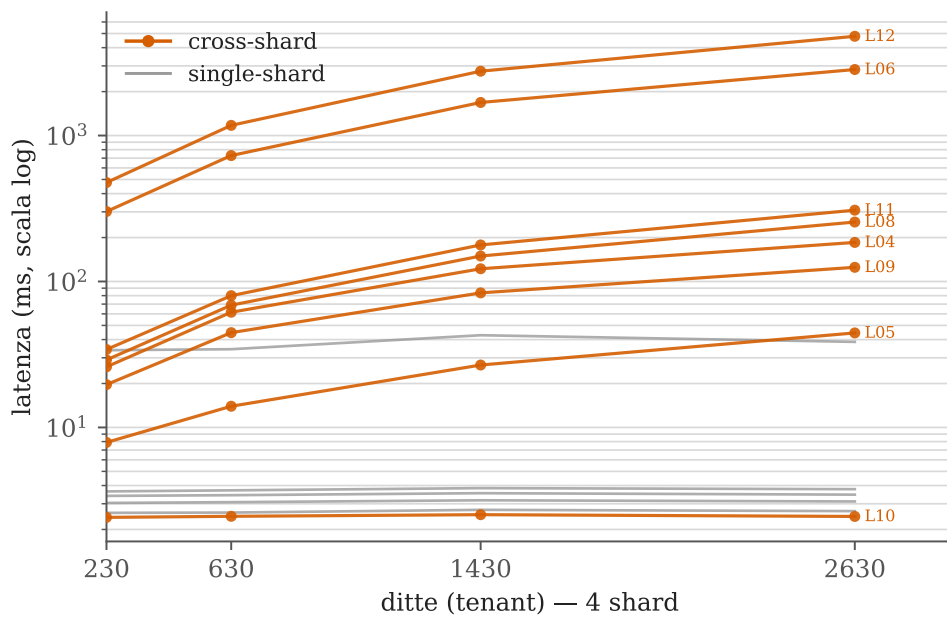


Figura 5.3: MongoDB, scaling per dati (tutte le 13 letture, 4 shard fissi, scala log): L06 e L12 crescono a dismisura col volume, le single-shard restano piatte.

```

1 export MONGO_PASSWORD=...
2 bash scala-dati-mongo.sh 64 200 400 800 # +200/+400/+800 tenant -> 230, 630, 1430 ditte

```

Listing 5.6: Esecuzione dello scaling per dati (MongoDB): 4 shard fissi, lotti crescenti di tenant, misura a caldo.

5.6.3 Scritture (MongoDB)

Le scritture (4 shard, N=300, tenant fisso) confermano la co-locazione e, su U03, producono il contrasto pro-relazionale più marcato dell'intero lavoro (Tabella 5.9). Le scritture *intra-tenant* (I01, I03, U01, U02, U04) colpiscono **un solo shard** (quello del tenant): il carico è tutto su 10.0.1.4 (CPU ~5%), gli altri tre inattivi, esattamente come in Citus. **U03** invece, che aggiorna una categoria *denormalizzata* in tutti i cedolini, è un **updateMany** che tocca **tutti e 4 gli shard al 91–99% di CPU**: 356 ms per operazione e appena 2.8 op/s. In Citus la stessa modifica è **una riga** su una reference table (21 ms, 47 op/s): un fattore ~17× a favore del relazionale.

Tabella 5.9: MongoDB, scritture a 4 shard (N=300): la co-locazione (una scrittura, uno shard) e il mass-update U03.

Op.	Descrizione	lat (ms/op)	tps	dove scrive
U02	licenziamento	3.6	279	solo 1 shard
U01	coordinate bancarie	3.6	279	solo 1 shard
U04	update concorrente	3.6	277	solo 1 shard
I01	inserimento timbratura	8.6	116	solo 1 shard
I03	elaborazione cedolino	11.1	90	solo 1 shard
U03	modifica dato di riferimento	355.9	2.8	tutti i 4 shard (91–99% CPU)

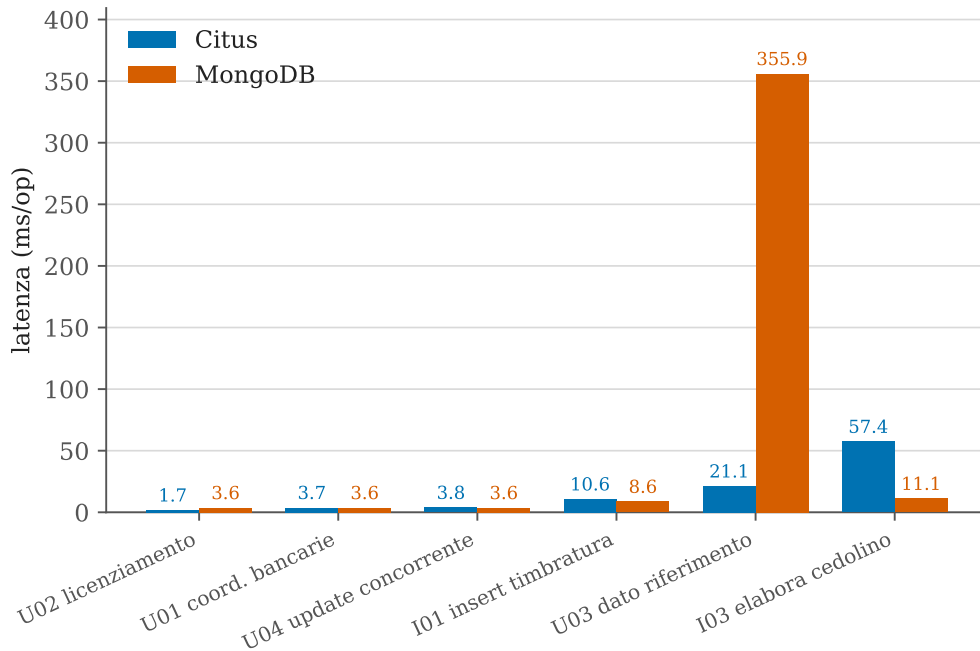


Figura 5.4: Scritture a confronto (4 nodi/shard, N=300): latenza per operazione, Citus vs MongoDB.

```
1 bash esegui-scritture-mongo.sh 300 queries/document/scritture/*.js
```

Listing 5.7: Esecuzione delle scritture (MongoDB): N=300 ripetizioni per query, tenant fisso.

5.6.4 Comportamento in presenza di guasti (MongoDB)

Con lo stesso protocollo del test di guasto Citus, si esegue L04 in loop mentre lo shard che ospita i dati del tenant (10.0.1.4) viene spento e riacceso (10 run). Il comportamento è **CP**: mentre lo shard è giù la query fallisce e le righe crollano da 59 299 a **0** (nessun risultato parziale), per tornare al valore pieno al rientro; il recupero è automatico (~ 0.25 s dopo il riavvio) e l'indisponibilità dura esattamente il tempo di down (~ 15 s) (Tabella 5.10).

Tabella 5.10: MongoDB, guasto di uno shard sotto carico di lettura (10 run).

Shard spento	run	error-rate	finestra / recupero	righe (su→giù→su)
10.0.1.4 (con i dati)	10	7.0 %	10.1→25.2 s / ~ 0.25 s	59 299 → 0 → 59 299

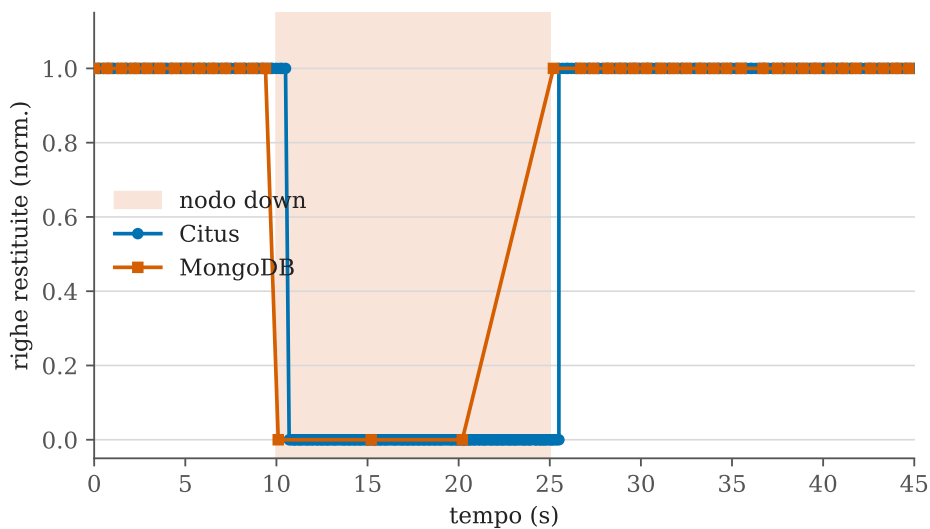


Figura 5.5: Guasto di un nodo sotto carico L04, Citus e MongoDB: righe restituite (normalizzate) nel tempo. In entrambi crollano a 0 durante il down (banda) e recuperano al rientro (comportamento CP).

Un dettaglio metodologicamente importante: l'error-rate è solo $\sim 7\%$, contro il $\sim 51\%$ di Citus, *pur avendo la stessa indisponibilità reale* (15 s). La ragione è il **diverso comportamento del client**: il mongos **si blocca in attesa** dello shard irraggiungibile fino al timeout (5 s per operazione), quindi nella finestra di down passano poche operazioni-errore “lunghe”; Citus invece **rifiuta la connessione all’istante** (fail-fast) e ne accumula molte di più. L'error-rate grezzo non è dunque direttamente comparabile: va letto insieme al *modo* di fallire, cioè **timeout bloccanti** in Mongo contro **errori rapidi** in Citus.

```
1 bash test-guasto-mongo.sh queries/document/letture/L04_gender_gap_mediano.js 10.0.1.4 10 15 45
```

Listing 5.8: Esecuzione del test di guasto (MongoDB): L04 in loop, arresto/riavvio dello shard con i dati.

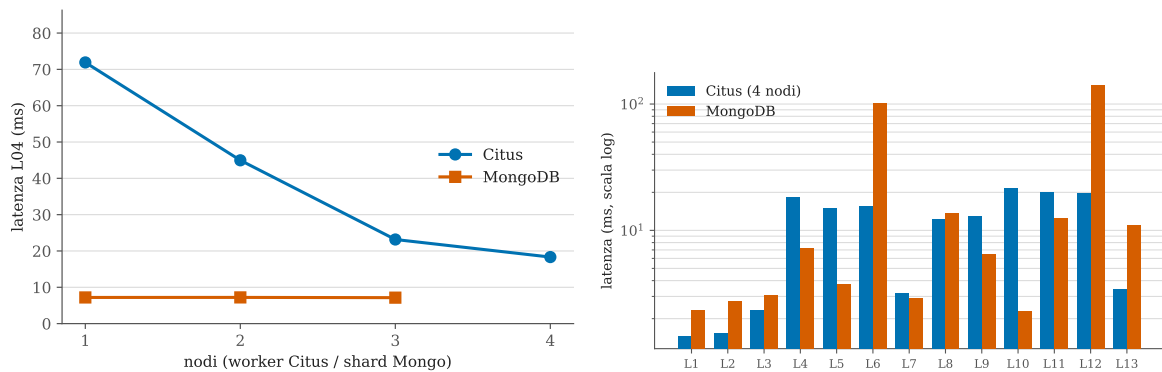
5.7 Confronto tra i due sistemi

Il confronto si articola su più assi (Tabelle 5.11 e 5.12):

- **Analitica:** dipende dal tipo di query. Dove l'aggregato è servito dall'embedding (percentili L04, spesa per categoria L11) MongoDB è più veloce, perché evita le join (a 1 nodo/shard L04 7.2 vs 71.9 ms, L11 12.4 vs 75.6 ms). Ma sulle query che **attraversano più entità** (assenteismo L06, ferie residue L12, riepilogo assenze L13, che toccano assenze e ratei) vince Citus, e di molto: a pari scala (~ 230 – 240 ditte, 4 nodi) L06 51 vs 302 ms, L12 28 vs 476 ms, L13 3.6 vs 34 ms. MongoDB paga lo scan degli array annidati sull'intera collezione, Citus sfrutta i join indicizzati sulle tabelle normalizzate.
- **Scaling:** Citus distribuisce e scala *sempre* (rebalance esplicito, $\sim 4\times$ con 4 nodi); MongoDB invece dipende dalla scala: a dati piccoli il balancer non distribuisce (nessuno scaling), a volume grande sì (tutti gli shard attivi, cfr. Sezione 5.6.2).
- **Scritture:** entrambi co-locano la scrittura di un tenant su un solo nodo/shard, ma sul dato di riferimento (U03) il relazionale è nettamente più efficiente (1 riga, 21 ms vs mass-update, 356 ms).
- **Guasti:** entrambi CP con la stessa indisponibilità reale, ma *modo* di fallire opposto (fail-fast vs timeout bloccante).
- **Query mirate** (cedolino singolo): comparabili (~ 1 – 2 ms) su entrambi.

Tabella 5.11: Confronto in latenza (ms) a pari numero di nodi (1) e capacità di scalare.

Query	Citus	MongoDB	Scaling coi nodi
L04 gender gap	71.9	7.2	Citus $\sim 4\times$; Mongo no (scala piccola)
L11 spesa/categoria	75.6	12.4	Citus $\sim 4\times$; Mongo no (scala piccola)
L01 cedolino singolo	1.5	2.3	entrambi piatti (mirata)



(a) Scaling per nodi su L04: Citus scala, Mongo piatto (scala piccola). **(b)** Latenza assoluta delle 13 letture a scala base (log).

Figura 5.6: Confronto Citus vs MongoDB: capacità di scalare coi nodi (a) e velocità assoluta in lettura (b).

Il confronto diretto sulle **scritture** (Tabella 5.13) dà esiti opposti a seconda dell'operazione: sull'inserimento del cedolino (I03) è molto più rapido MongoDB (11 vs 57 ms), perché scrive un solo documento invece di una transazione su sette tabelle; sull'aggiornamento del dato di riferimento (U03) è molto più rapido Citus (21 vs 356 ms), perché tocca una riga su una reference table invece di aggiornare in massa tutti gli shard. Le scritture semplici (U01/U02/U04) sono equivalenti. Sull'**analitica** (Tabella 5.11) MongoDB è più veloce in assoluto grazie all'embedding.

Tabella 5.12: Confronto complessivo Citus vs MongoDB sui diversi assi misurati.

Aspetto	PostgreSQL + Citus	MongoDB
Distribuzione dei dati	esplicita (rebalance , deterministica)	automatica (balancer, soggetta a soglia)
Scaling per nodi (dati piccoli)	$\sim 4\times$ con 4 nodi (cross-shard)	nessuno (balancer sotto soglia)
Scaling per dati (volume grande)	cross-shard super-lineari	idem, e il balancer distribuisce
Analitica sull'aggregato (L04, L11)	più lenta (join)	più veloce (embedding)
Query multi-entità (L06, L12, L13)	più veloce (join indicizzati)	molto più lenta (array annidati)
Scrittura intra-tenant	1 nodo (co-locazione)	1 shard (co-locazione)
Dato di riferimento (U03)	1 riga: 21 ms, 47 op/s	mass-update: 356 ms, 2.8 op/s
Guasto (modo di fallire)	fail-fast (err $\sim 51\%$)	timeout bloccante (err $\sim 7\%$)
Guasto (indisponibilità reale)	~ 15 s, recupero ~ 0.5 s	~ 15 s, recupero ~ 0.25 s

Tabella 5.13: Scritture a confronto (4 nodi/shard, N=300): latenza e throughput per operazione.

Op.	Descrizione	Citus		MongoDB	
		ms/op	op/s	ms/op	op/s
U02	licenziamento	1.7	605	3.6	279
U01	coordinate bancarie	3.7	273	3.6	279
U04	update concorrente	3.8	266	3.6	277
I01	inserimento timbratura	10.6	94	8.6	116
I03	elaborazione cedolino	57.4	17	11.1	90
U03	dato di riferimento	21.1	47	355.9	2.8

Il contrasto tra i modelli si vede meglio sulle coppie deliberate. Il **cedolino completo** è pro-documentale: una `findOne` su un documento (Elenco 5.9) contro l'assemblaggio da sette tabelle in Citus. Il **dato di riferimento** è pro-relazionale: una riga su una reference table (Elenco 5.10) contro un `updateMany` di centinaia di documenti denormalizzati in Mongo.

```
1 db.cedolini.findOne({ tenant_id: 1, dipendente_id: 7, anno: 2026, mese: 6 });
```

Listing 5.9: Cedolino completo in MongoDB: un solo documento (pro-documentale).

```
1 UPDATE tipo_voce SET categoria = 'WELFARE' WHERE codice = '0401'; -- 1 riga (in Mongo:
updateMany su N documenti)
```

Listing 5.10: Modifica di un dato di riferimento in Citus: una riga (pro-relazionale).

6 Conclusioni

Il lavoro ha confrontato, sullo stesso dominio HR/retributivo e sulla stessa infrastruttura, un relazionale distribuito (PostgreSQL+Citus) e un documentale (MongoDB), entrambi CP ma con modelli di dati opposti. Ne emerge che la domanda “quale sistema è migliore” non ha risposta assoluta: dipende dall’operazione e dalla scala, coerentemente con l’idea che non esista una taglia unica [2].

6.1 Sintesi dei risultati

- **Scalabilità per nodi (Citus):** le analitiche cross-shard scalano quasi linearmente ($\sim 3.8\text{--}4.6\times$ con 4 nodi); le query mirate a un tenant restano piatte perché colpiscono un solo shard, ma sono già nell’ordine del millisecondo. Il miglioramento è più marcato proprio per le query che aggregano e uniscono (join) dati di più tenant, come mostrano Tabelle 5.1 e 5.2: la L05, ad esempio, arriva a $4.4\times$ passando da 1 a 4 nodi.
- **Scalabilità per dati (Citus):** a nodi fissi le cross-shard degradano super-linearmente col volume, mentre le single-shard restano insensibili grazie all’indicizzazione e alla co-localizzazione per tenant.
- **Scritture (Citus):** la co-localizzazione fa sì che la scrittura di un tenant colpisca un solo nodo; solo la modifica di un dato di riferimento tocca tutti i nodi. La transazione multi-tabella è l’operazione più costosa.
- **Guasti:** comportamento CP confermato; con un nodo giù la query cross-shard fallisce per l’intera finestra (nessun risultato parziale), con recupero automatico e rapido al rientro. La consistenza è scelta al posto della disponibilità.
- **MongoDB:** più veloce sulle analitiche che l’embedding serve direttamente (percentili, spesa per categoria), ma **nettamente più lento** sulle query che attraversano più entità (assenteismo, ferie residue, che toccano assenze e ratei): lì Citus, con i join indicizzati, è risultato più rapido di 6–17 volte a pari scala. Inoltre il balancer, volutamente conservativo, **non distribuisce a scala piccola**, quindi a quella scala aggiungere shard non porta beneficio; la distribuzione riparte solo a volume grande, in contrasto con il rebalance esplicito e deterministico di Citus.
- **Modellazione:** il cedolino completo premia il documentale (un documento vs sette tabelle), mentre il dato di riferimento e le aggregazioni multi-entità premiano il relazionale (una riga vs aggiornamento di massa; join indicizzati vs scan di array annidati).

6.2 Quando conviene ciascun sistema

Sulla base delle misure: **Citus** è preferibile quando servono analitiche cross-tenant che devono *scalare* orizzontalmente in modo prevedibile e, soprattutto, per le **query complesse che attraversano più entità** (aggregazioni e join profondi su assenze, ratei, contratti): proprio lì, dove MongoDB deve scorrere gli array annidati sull’intera collezione, Citus è risultato molte volte più veloce a pari scala. Si aggiungono l’integrità referenziale forte, le transazioni distribuite e il controllo esplicito della distribuzione tipico di un DBA. **MongoDB** è preferibile quando il carico è dominato da operazioni sull’*aggregato* (lettura/scrittura del

documento completo), quando conta la bassa latenza sulle singole richieste e la flessibilità di schema, accettando in cambio un controllo meno fine sulla distribuzione e la denormalizzazione dei dati condivisi.

6.3 Limiti e sviluppi futuri

I risultati vanno letti alla luce di alcuni limiti dichiarati. La scala è dell'ordine dei GB (vincolo del disco delle VM), non delle centinaia di GB. Il working set è sempre rimasto in RAM (cache di Postgres e di WiredTiger): le letture da disco sono state nulle, quindi il regime I/O-bound non è stato raggiunto. Gli esperimenti si sono fermati quando la tendenza era già chiara e i dati risultavano distribuiti su tutti i nodi, senza spingersi fino a saturare il disco. Entrambi i sistemi sono inoltre a copia singola (nessuna replica), scelta voluta per un test di guasto CP corretto ma che esclude l'alta disponibilità.

Come sviluppi: spingere i volumi fino alla saturazione effettiva del disco per osservare il regime I/O-bound; introdurre la replica per confrontare disponibilità e comportamento in failover; e misurare esplicitamente uno scenario di evoluzione di schema (turni/ferie) su entrambi i sistemi, oggi trattato solo a livello teorico.

Bibliografia

- [1] P. Atzeni et al. *Basi di dati: Modelli e linguaggi di interrogazione*. 5^a ed. McGraw-Hill Education, 2018.
- [2] M. Stonebraker e U. Çetintemel. «“One Size Fits All”: An Idea Whose Time Has Come and Gone». In: *Proceedings of the 21st International Conference on Data Engineering (ICDE)* (2005), pp. 2–11. DOI: 10.1109/ICDE.2005.1.
- [3] Citus Data / Microsoft. *Citus Documentation*. <https://docs.citusdata.com/>. consultato nel 2026. 2024.
- [4] MongoDB, Inc. *MongoDB Manual*. <https://www.mongodb.com/docs/manual/>. consultato nel 2026. 2024.
- [5] S. Gilbert e N. Lynch. «Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services». In: *ACM SIGACT News* 33.2 (2002), pp. 51–59. DOI: 10.1145/564585.564601.
- [6] D. J. Abadi. «Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story». In: *Computer* 45.2 (2012), pp. 37–42. DOI: 10.1109/MC.2012.33.